Qubit-Label-Invariant Dependency Graphs for Out-of-Distribution Quantum-Circuit Learning

Anonymous Authors
Affiliation withheld for review

November 6, 2026

Abstract

Quantum-circuit models are commonly trained on fewer qubits than target workloads, yet many encoders contain absolute wire-index features that impose a fixed width and change under arbitrary logical-qubit renaming. We introduce a qubit-label-invariant circuit encoder: nodes contain only gate type, angle, and arity, while directed dependency edges encode whether their shared wire is a single-qubit operand, CX control, or CX target at each endpoint. This representation is exactly invariant to any permutation of logical-qubit labels and accepts widths unseen during training. We compare an 80,833-parameter six-layer dependency GNN with an 84,961-parameter canonical-grid GRU using identical index-free node features. Models train on structured QAOA-like, VQE, and arithmetic circuits with 4–7 qubits and test on circuits up to 20 qubits. Across five frozen paired seeds, the GNN reaches 0.9252 AUROC and 0.8458 F1 on 13–20-qubit circuits versus 0.8807 and 0.8011 for the grid; the GNN wins every seed by 0.0417–0.0484 AUROC. Logical-qubit relabeling changes GNN logits by exactly zero, while grid AUROC falls by 0.1097. A fixed post-confirmatory canonical Transformer reaches 0.7471 far-width AUROC. The results show that enforcing a physically irrelevant symmetry through dependency roles improves within-family out-of-distribution generalization. Unseen-family and practical compiler generalization remain open.

1 Introduction

Quantum-circuit learning has an unavoidable scale mismatch. Exact simulation, compiler search, and labeled-data generation are easiest on small circuits, while useful workloads contain more qubits and operations. A learned circuit representation must therefore extrapolate beyond its training width rather than only interpolate among familiar shapes.

Absolute logical-qubit labels should not determine a structural prediction. Relabeling qubits by a permutation changes symbols such as q_0 and q_7 but preserves gate types, shared-wire dependencies, circuit depth, and critical-path membership. Nevertheless, common tensor and sequence encodings allocate one feature slot or grid row per wire. They have a fixed maximum width and do not guarantee identical predictions after relabeling.

We study whether encoding this symmetry improves out-of-distribution (OOD) learning. Our diagnostic task is depth-critical gate prediction: identify operations lying on at least one maximum wire-dependency path. Critical paths are exactly computable, so this is not a learned replacement for the algorithm. Exact labels remove simulator and oracle noise and isolate how representation affects relational reasoning.

Our contributions are:

1. a circuit DAG whose node and edge features contain no absolute qubit identity and whose per-operation predictions are exactly invariant under every logical-qubit permutation;
2. a frozen five-seed comparison that trains on 4–7 qubits and tests on structured circuits with 13–20 qubits, beyond the original encoder's 12-wire feature ceiling;
3. near-matched canonical-grid GRU and canonical Transformer controls using the same index-free node features;
4. a separation between within-family width extrapolation, exact relabeling symmetry, and the still-unresolved problems of unseen-family and practical compiler generalization.

2 Symmetry-aware circuit representation

2.1 Wire-dependency graph

Let a circuit contain ordered operations $V = \{v_i\}_{i=1}^{m}$ on logical qubits Q. For each $q \in Q$, connect consecutive operations that touch q. The union of these directed edges is a DAG $G = (V, E)$. Independent operations have no edge even when one happens to be serialized first.

Every node has seven features: a four-way gate type (H, RY, RZ, or CX), sine and cosine of its rotation angle, and normalized arity. No feature identifies a wire number. For an edge induced by shared wire q, each endpoint receives one of three roles: single-qubit operand, CX control, or CX target. Concatenating source and destination

roles gives a six-dimensional edge attribute.

This construction adapts sparse LayerDAG-style aggregation [1] to quantum-circuit dependencies. Six residual blocks pass messages in both dependency directions. A message concatenates the sender state with the six role features. A linear head emits one logit per operation.

2.2 Exact qubit-label invariance

Let $\pi : Q \to Q$ be any permutation of logical-qubit labels. Relabeling every operand by π preserves whether two operations share a wire and preserves the operand role at each endpoint. Node features, edge incidence, and edge-role attributes are therefore unchanged. The message-passing computation and every per-operation logit are identical:

$$f_{\mathrm{DAG}}(\pi(C)) = f_{\mathrm{DAG}}(C). \tag{1}$$

This is architectural invariance, requiring neither data augmentation nor a learned approximation. It follows the geometric-learning principle of matching model symmetry to task symmetry [2, 3].

2.3 Matched controls

The **canonical-grid GRU** receives the same seven-dimensional node features. It computes deterministic earliest dependency layers, orders gates by operands and type, and applies a two-layer bidirectional GRU. It does not ingest an absolute index feature, but qubit relabeling can change its canonical order, so invariance is not guaranteed.

The **canonical Transformer** uses the same canonical order and node features, width 64, two encoder layers, four heads, feed-forward width 160, learned positions, and no dropout. It is a fixed post-confirmatory robustness control rather than part of the predeclared DAG-versus-grid gate.

Parameter counts are 80,833 for DAG, 84,961 for grid, and 88,065 for Transformer.

3 Controlled prediction task

For node v, let $a(v)$ be the longest predecessor path ending at v, $b(v)$ the longest successor path beginning at v, and $D = \max_v a(v)$. The exact label is

$$y_v = \mathbf{1}[a(v) + b(v) - 1 = D]. \tag{2}$$

The label depends on global wire structure rather than gate type alone. It is invariant to legal operation reordering and logical-qubit relabeling.

The task is a representation probe. A production compiler should calculate critical paths directly. Here the exact algorithm supplies noise-free supervision for testing OOD inductive bias.

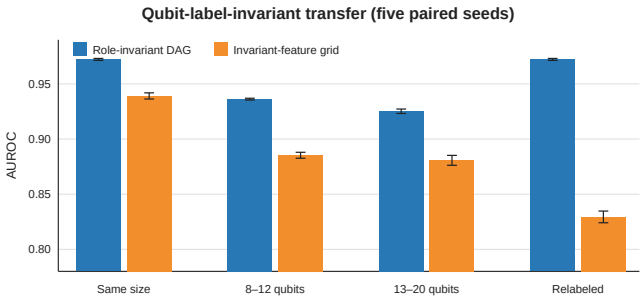


Figure 1: Qubit-label-invariant final results. The DAG transfers from 4–7 training qubits to 13–20 test qubits and is unchanged by logical-qubit relabeling. Error bars show sample standard deviation across five paired seeds.

4 Experimental design

4.1 Data and shifts

Circuits are balanced across QAOA-like [4], hardware-efficient VQE [5], and reversible linear-arithmetic generators. Training contains 1,000 circuits with 4–7 qubits and 32–64 gates; validation contains 256 fresh circuits. Four final splits contain 512 circuits each:

- **same size**: 4–7 qubits and 32–64 gates;
- **near width**: 8–12 qubits and 72–104 gates;
- **far width**: 13–20 qubits and 96–144 gates;
- **relabeled**: the same-size circuits after independent random permutations of logical-qubit labels.

All content hashes are stored with the results.

4.2 Training and firewall

DAG and grid train with paired seeds 111–115, identical circuits and minibatch indices, AdamW at 3×10^{-4}, batch size 16, gradient clipping at 1.0, and 4,000 updates. Checkpoints use validation AUROC only. No final split affects training or checkpoint selection.

A separate development seed fixed the architecture and ranges before fresh final data. The final claim required DAG to beat grid in far-width AUROC for all five seeds, mean gain at least 0.03, positive mean F1 gain, maximum relabeling logit difference at most 10^{-6}, and identical same-versus-relabeled DAG metrics. The Transformer architecture and budget were fixed after the primary result opened; it is reported descriptively and was not tuned after inspection.

We report node-level AUROC, F1, and accuracy. Means and sample standard deviations are over training seeds.

Table 1: Five-seed AUROC (mean ± sample standard deviation). Transformer is a post-confirmatory control.

Split	Role DAG	Grid GRU	Transformer
Same size	$\mathbf{.9722 \pm .0009}$	$.9391 \pm .0028$	$.8888 \pm .0010$
+8–12 qubits	$\mathbf{.9362 \pm .0008}$	$.8853 \pm .0027$	$.7467 \pm .0070$
+13–20 qubits	$\mathbf{.9252 \pm .0020}$	$.8807 \pm .0045$	$.7471 \pm .0128$
Relabeled	$\mathbf{.9722 \pm .0009}$	$.8294 \pm .0053$	$.7806 \pm .0064$

5 Results

5.1 Far-width transfer

The predeclared gate passes. DAG-minus-grid far-width AUROC differences are +0.0419, +0.0419, +0.0417, +0.0482, and +0.0484; mean +0.0444. The descriptive seed-level 95% t interval is [+0.0400, +0.0488]. DAG far-width F1 is 0.8458 versus 0.8011 for grid, a +0.0446 gain. The DAG retains 95.2% of its same-size AUROC at far width, while grid retains 93.8%.

The canonical Transformer is weaker on every split and reaches 0.7471 far-width AUROC. Because this control was specified after opening the primary result, it strengthens baseline coverage but is not confirmatory evidence.

5.2 Logical-qubit relabeling

DAG's maximum logit difference across 320 final circuit/relabel pairs is exactly 0.0, and AUROC, F1, and accuracy are identical before and after relabeling. Grid AUROC falls from 0.9391 to 0.8294, a 0.1097 drop. Transformer falls from 0.8888 to 0.7806.

The controls receive no absolute qubit-index feature, yet their canonical token order depends on labels. Removing explicit indices is therefore insufficient; the computation itself must respect the symmetry.

6 Relation to earlier size-transfer evidence

Before the invariant redesign, a fixed-width encoder with absolute wire slots was tested on random and structured circuits up to 12 qubits. DAG beat grid on every larger-circuit seed across a three-seed random study and a five-seed structured replication. Separating axes gave +0.0514 AUROC for width shift and +0.0025 for depth-only shift. These frozen studies motivated removing absolute labels and extending evaluation to 20 qubits. They are supporting evidence rather than pooled with the invariant final because the architectures differ.

A held-out chemistry-inspired family in the earlier structured study favored grid AUROC and left both models weak. Three rewrite-ranking follow-ups also failed to establish production optimization superiority. These negative results bound the claim: qubit-label symmetry and within-family width transfer do not imply unseen-family or compiler-policy generalization.

7 Related work

Graph neural networks encode relational inductive bias through message passing [6, 7, 2]. Geometric deep learning emphasizes architectures that respect known group actions [3]. LayerDAG develops layerwise autoregressive generation for generic DAGs [1]; our model instead predicts circuit-node labels using typed wire-dependency edges.

Quantum-machine-learning software supports hybrid circuit models [8], while quantum compilation must manage depth, two-qubit gates, and routing [9, 10, 11]. Our work addresses an earlier representation question: whether the circuit encoder preserves physically irrelevant qubit-label symmetry and extrapolates beyond training width.

8 Limitations and reproducibility

The benchmark is deterministic and synthetic, uses four gate types, and tests one exactly computable topological label. QAOA-like, VQE, and arithmetic generators provide structured diversity but are not a public application corpus. The invariant final does not test unseen families, hardware noise, device connectivity, or an expensive compiler objective. The Transformer receives one fixed matched configuration rather than a broad independent tuning sweep.

Every protocol predates its corresponding final result. The artifact stores generators, dataset hashes, five-seed histories, negative studies, tests, and compute accounting. The implementation is native PyTorch and does not claim runtime parity with upstream DGL LayerDAG. No GPU or paid resource is used.

9 Conclusion

Absolute qubit labels are a nuisance variable for structural circuit prediction. Encoding only operand roles on wire-dependency edges gives exact label invariance and supports widths beyond the training representation. Across five paired seeds, the resulting DAG outperforms matched recurrent and attention controls after extrapolating from 4–7 to 13–20 qubits. The result establishes a symmetry-aware inductive bias for within-family quantum-circuit OOD learning. Family shift and practical compiler utility remain separate open problems.

References

[1] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas Sridharan, Ying Zhang,

Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model for directed acyclic graph generation. In *International Conference on Learning Representations*, 2025.

[2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.

[3] Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*, 2021.

[4] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A quantum approximate optimization algorithm. *arXiv preprint arXiv:1411.4028*, 2014.

[5] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5:4213, 2014.

[6] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1263–1272, 2017.

[7] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[8] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Philip Massey, Ramin Halavati, Murphy Yuezhen Niu, Alexander Zlokapa, Evan Peters, Owen Lockwood, Andrea Skolik, Sofiene Jerbi, Vedran Dunjko, Martin Leib, Michael Streif, David Von Dollen, Hongxiang Chen, Shuxiang Cao, Roeland Wiersema, Hsin-Yuan Huang, Jarrod R. McClean, Ryan Babbush, Sergio Boixo, Dave Bacon, Alan K. Ho, Hartmut Neven, and Masoud Mohseni. Tensorflow quantum: A software framework for quantum machine learning. *arXiv preprint arXiv:2003.02989*, 2021.

[9] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.

[10] Gushu Li, Yufei Ding, and Yuan Xie. Tackling the qubit mapping problem for NISQ-era quantum devices. *ASPLOS*, pages 1001–1014, 2019.

[11] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. On the qubit routing problem. *Proceedings of the 14th Conference on the Theory of Quantum Computation, Communication and Cryptography*, 2019.